CHAPTER 1 + CHAPTER 2 · COMPLETE COVERAGE · BUILT TO MEMORIZE

SOFTWARE REQUIREMENTS ENGINEERING

— Master Study Guide —

Every concept from *The Essential Software Requirement* and *Requirements from the Customer & Analyst Perspective* — explained in plain English, tied to one running example, and wired for memory.

12 recreated figures · 20+ mnemonics & memory hooks · comparison tables · quick-recall boxes

one-look cheat sheet · 39-question self-test with answer pointers

Compiled July 2026 · from the course slide decks (Wiegers & Beatty, *Software Requirements*, 3rd ed.) · running example “QuickBite” is fictional

Contents

PART 1 — What Requirements Are 5

1.1 When requirements go wrong: the 10 classic problems 5
1.2 What is a “requirement”? 5
1.3 The three levels: business, user, functional 6
1.4 System requirements 7
1.5 Non-functional requirements (quality attributes) 8
1.6 Business rules 8
1.7 Features 9
1.8 How all the requirement types connect 10
1.9 Product vs. project requirements 10

PART 2 — The Requirements Engineering Process 12

2.1 The big picture: development + management 12
2.2 Requirements development: ELICITATION 12
2.3 Requirements development: ANALYSIS 13
2.4 Requirements development: SPECIFICATION 13
2.5 Requirements development: VALIDATION 13
2.6 The framework is iterative, not linear 14
2.7 Requirements MANAGEMENT 15
2.8 Why requirements matter — Brooks’ warning 16
2.9 Reasons behind bad requirements 16
2.10 Benefits of a high-quality requirements process 17

PART 3 — Stakeholders, Customers & the Partnership 19

3.1 The expectation gap 19
3.2 Stakeholders: the full cast 19
3.3 Who is the customer? (and who are users?) 20
3.4 The customer’s nine RIGHTS 21
3.5 The customer’s nine RESPONSIBILITIES 21
3.6 Making decisions & reaching agreement 22
3.7 The requirements baseline — and the confidence it buys 23

PART 4 — The Business Analyst 24

4.1 The BA role: a communication bridge 24
4.2 The BA's ten tasks 24
4.3 Essential analyst SKILLS (all 14) 25
4.4 Essential analyst KNOWLEDGE 25
4.5 The making of a business analyst — four paths 26

PART 5 — Rapid Revision 27

5.1 One-look cheat sheet 27
5.2 Practice question bank 28

How to use this guide

The guide is **topic-organized**, not slide-organized: related slides are merged into one section each, and the grey chip on every heading (e.g. **CH-1 · SLIDE 12**) tells you exactly which slides it covers — so you can always cross-check the deck. Nothing from the decks is skipped except title, outline and reference slides.

Every topic follows the same rhythm: a plain-English explanation → the deck's own example → the same idea inside one running example → a one-line memory hook. The colored boxes mean:

DEFINITION — Term

The precise meaning you should be able to write in an exam — faithful to the slide wording.

EXAMPLE — FROM THE SLIDES

The example the slides themselves use (airline kiosk, supermarket workstation, warehouse email report…) — quote these to show you've studied the source.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

The same concept inside **QuickBite**, a fictional food-delivery app that runs through the whole guide. Meeting every abstract idea inside one familiar story is what makes the ideas stick — and because it's one product, you can see how business requirements, rules, constraints and features differ *on the same app*.

MEMORY HOOK A one-line handle — a mnemonic, a picture, or a rhythm. If you remember nothing else from a section, remember this line.

EXAM TRAP

A place where exams love to catch people — usually a direction or a distinction that is easy to flip.

QUICK RECALL

- End-of-part checklists. Cover them, recite, uncover, check.

Suggested three passes: (1) read Parts 1–4 in full, redrawing each figure as you meet it; (2) skim only the amber hooks and Quick Recall boxes; (3) close the book and work through the question bank in Part 5, following the answer pointers for anything shaky.

PART 1

What Requirements Are

Chapter 1 · The essential software requirement — definitions, levels, and every type of requirement

1.1 When requirements go wrong: the 10 classic problems

CH-1 · SLIDES 2-3

Projects rarely fail at the keyboard — they fail earlier, in the requirements. The slides open with ten problems that show up again and again. Memorize them as a **story in four stages**:

Stage	The problems (memorize all ten)
Stage A — Foggy start	1. Business objectives, vision, and scope never clearly defined. 2. Lack of communication — customers “too busy” to work with analysts/developers on what users needed. 3. Customers claim *everything* is critical, so nothing gets prioritized.
Stage B — Blind build	4. Developers hit ambiguities and missing information while coding, so they **guess**. 5. Customers never approved the requirements.
Stage C — Change chaos	6. Customers approved requirements for a release/iteration, then changed them continually. 7. Scope grew as changes were accepted, but the schedule slipped — no extra resources given, no functionality removed. 8. Change requests got lost; nobody knew the status of any request.
Stage D — Hollow finish	9. Customers requested functionality, developers built it — and nobody ever uses it. 10. The specification was satisfied, but the customer and the business objectives were **not**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

QuickBite’s first attempt: the COO never wrote down the goal (foggy start); restaurant owners were too busy for interviews, so developers guessed how menus work (blind build); every week a new “critical” change arrived while the deadline stayed fixed (change chaos); launch day delivered a coupon engine nobody opens, and phone-order costs stayed exactly the same (hollow finish).

MEMORY HOOK **Foggy start → Blind build → Change chaos → Hollow finish.** Four stages, counts 3–2–3–2 = the 10 problems.

1.2 What is a “requirement”?

CH-1 · SLIDE 4

DEFINITION — Requirement

A property a product **must have to provide value to a stakeholder**. Requirements are a **specification of what should be implemented** — descriptions of a system property or attribute, or of how the system should behave. They may also be a **constraint** on the development process.

Two ideas inside this definition are exam favourites:

- **Two viewpoints.** Requirements encompass the **user's view of external system behaviour** (what I see it do) *and* the **developer's view of internal characteristics** (what it must be like inside).
- **A time dimension.** Requirements can be **present tense** (what the current system does), **future** — near-term (high priority) or hypothetical (low priority) — or even **past tense** (needs once specified, then discarded).

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

External view: "when I tap Reorder, my last meal is in the cart." Internal view: "order history must be cached locally for offline reads." Tense: "app shows live rider location" (present, v1), "support scheduled orders" (near-term future), "drone delivery" (hypothetical), "fax-order support" (past — discarded).

MEMORY HOOK Requirement = **VALUE to a stakeholder**, seen from **2 views** (outside/inside), living in **3 tenses** (past, present, future).

1.3 The three levels: business, user, functional

CH-1 · SLIDES 5-9

Software requirements exist on **three distinct levels**, and keeping them **aligned** is essential for project success. Learn each level as an answer to a different question:

Level	Meaning, source and form
Business requirements *WHY are we building this?*	Describe **why the organization is implementing the system** — the business benefits it hopes to achieve. Focus is on the objectives of the organization or the customer requesting the system. They typically come from the **funding sponsor, acquiring customer, manager of the users, marketing department, or a product visionary**. Warning: business requirements collected from multiple sources **might conflict**.
User requirements *WHAT can a user DO with it?*	Describe **goals or tasks users must be able to perform** with the product that provide value to someone — plus product attributes/characteristics important to **user satisfaction**. Represented as **use cases** and **user stories**; the information comes from **user representatives**.
Functional requirements *WHAT must developers BUILD?*	Specify the **behaviours the product will exhibit under specific conditions** — what developers must implement to enable the user tasks (user requirements), thereby satisfying the business requirements. Traditionally written as **"shall" statements**.

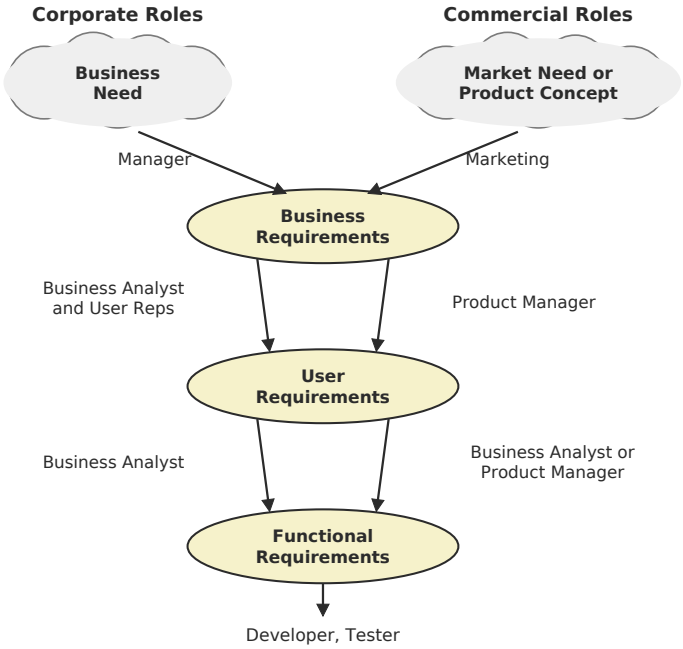
EXAMPLE — FROM THE SLIDES

One airline story ties all three together. **Business:** the airline wants to **cut airport counter-staff costs by 25%** — which sparks the idea of a self-service check-in **kiosk** (self-service technologies, SSTs, are used across many businesses). **User:** the use case "**Check in for a flight**" — or as a user story, "As a passenger, I want to check in for a flight so I can board my airplane." **Functional:** "The Passenger shall be able to print boarding passes for all flight segments he has checked in for" and "If the Passenger's profile has no seating preference, the reservation system shall assign a seat."

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Business: "cut call-center order-taking cost 40% and grow completed orders 30% within 12 months" (from the COO — the funding sponsor). **User:** "As a hungry customer, I want to reorder my last meal in one tap so I get food without searching." **Functional:** "The app shall show a Reorder button for the most recent completed order; when tapped, the system shall rebuild the cart within 2 seconds."

Who turns which level into which? The figure below (from the slides) shows how different stakeholders participate:



A business need (via a manager) or a market need/product concept (via marketing) becomes business requirements; a business analyst with user reps — or a product manager — derives user requirements; the BA (or product manager) derives functional requirements for developers and testers.

EXAM TRAP

A **constraint** is *not* a behaviour: it is a **restriction imposed on the choices available to the developer** for design and construction. Slide example: "The system shall be developed using open source tools and shall run on Linux OS." QuickBite: "must run on Android 8+." Constraints appear at the functional-requirements slide but are their own category in the relationships map (§1.8).

MEMORY HOOK **WHY → WHAT (user) → BUILD (developer)**. Business = why, User = what the user does, Functional = what the developer implements. Alignment of all three = project success.

1.4 System requirements

CH-1 · SLIDE 10

DEFINITION — System requirements

Requirements for a product **composed of multiple components or subsystems**. A system can be all software, or include **both software and hardware** subsystems (e.g., biometric devices). **People and processes are part of a system**, so certain system functions might be allocated to humans.

EXAMPLE — FROM THE SLIDES

The **supermarket cashier's workstation**: a bar-code scanner integrated with a scale, a hand-held bar-code scanner, a keyboard, a display, and a cash drawer. The requirements for the system as a whole lead the business analyst to **derive functionality allocated to each subsystem** — and demand an understanding of the **interfaces between them**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

The QuickBite "system" = customer app + rider app + restaurant tablet + payment gateway + dispatch service — plus **human riders and support staff** (people are part of the system: "verify cash payment" is allocated to the rider, not to code). The BA must specify the interfaces, e.g., the order-status messages between tablet and dispatch service.

MEMORY HOOK System = **sum of subsystems + people + interfaces**. Picture the checkout counter: scanner, scale, drawer… and the cashier herself.

1.5 Non-functional requirements (quality attributes)

CH-1 · SLIDE 11

DEFINITION — Non-functional requirement (quality attribute)

A product requirement that describes a **service or performance characteristic** — the famous "**-ity / -ilities**." They describe the product's characteristics in dimensions important to users *or* to developers and maintainers.

The slide packs four distinct points — all quizzable:

- **Dimensions:** performance, safety, availability, portability … and note: the chance of **conflicts among non-functional requirements is fairly high** (e.g., tighter security usually hurts usability).
- **External interfaces:** other NFR classes describe interfaces between the system and the outside world (usefulness, flexibility, reliability) — connections to other software, hardware components, users, and **communication interfaces (interoperability)**.
- **Design & implementation constraints:** restrictions on the developer's options during construction (the slide files **security** here).

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Performance: "restaurant list loads in ≤3 s on 3G." Availability: "99.9% up during dinner hours." Usability: "rider app operable one-handed." Interoperability: "talks to the bKash payment gateway API." Conflict example: end-to-end payment encryption (security) vs. one-tap checkout (usability) — the BA must negotiate.

MEMORY HOOK NFR = **HOW WELL**, not what. Hear "-ility" → think quality attribute; and remember: **-ilities love to fight each other**.

1.6 Business rules

CH-1 · SLIDE 12

DEFINITION — Business rule

Corporate policies, government regulations, industry standards, and computational algorithms. Business rules are **not themselves software requirements**, because they **exist beyond the boundaries of any specific software application**.

How rules relate to requirements — three chains to remember:

- Rules often **dictate that the system must contain functionality to comply** (slide example: an **online purchase limit**).
- Rules can impose **restrictions on the developer's design choices** — the slide's example "**no red colour in the webpage**" is called an **inverse requirement** (something the system must *not* do/have).
- Rules (e.g., **corporate security policies**) can be the **origin of quality attributes** that are then implemented in functionality (slide example: a **firewall**). So you can **trace certain functional requirements back to a business rule**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Regulation: "only restaurants with a valid hygiene license may operate" exists whether or not QuickBite exists — but it forces a license-verification feature. Policy: "max 3 pending cash-on-delivery orders per account" → an order-limit check. Security policy → OTP login (quality attribute → functionality). Inverse requirement: "never show competitor ads."

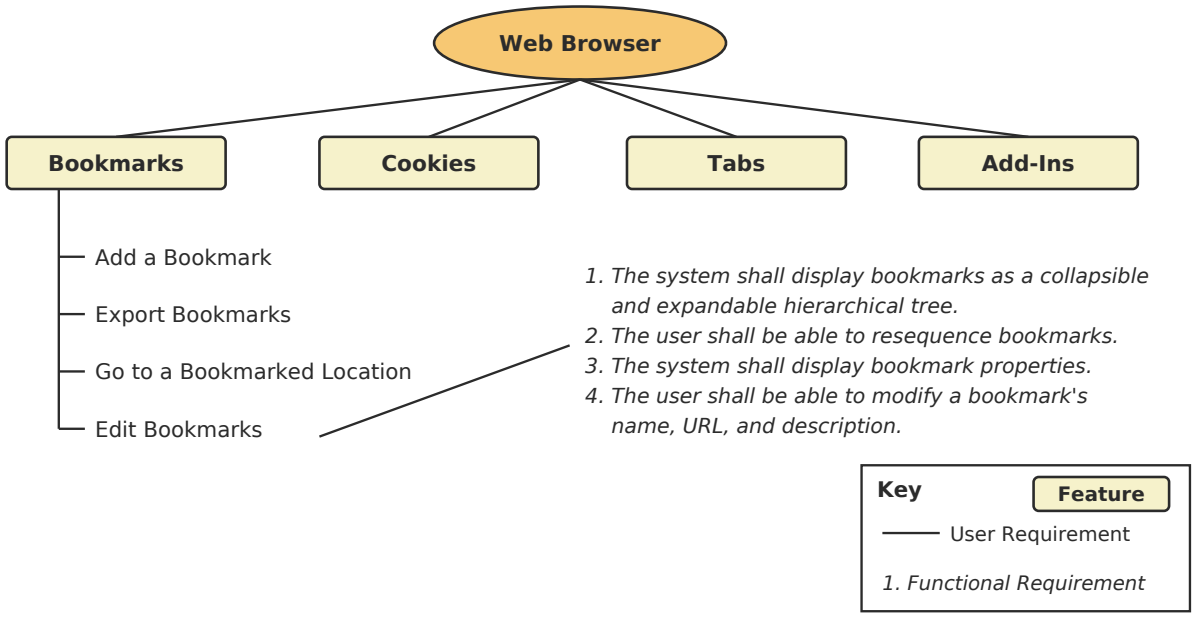
MEMORY HOOK Business rules **live OUTSIDE the software** but **reach INSIDE** it: rule → functionality, rule → constraint, rule → quality attribute → functionality.

1.7 Features

CH-1 · SLIDE 13

DEFINITION — Feature

One or more logically related system capabilities that provide value to a user and are **described by a set of functional requirements**. A feature can encompass **multiple user requirements**, each of which implies that certain **functional requirements** must be implemented so the user can perform that task.



The slides' web-browser example: the product has features (Bookmarks, Cookies, Tabs, Add-Ins); the Bookmarks feature contains user requirements (Add / Export / Go to / Edit a bookmark); "Edit Bookmarks" alone implies four functional requirements.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

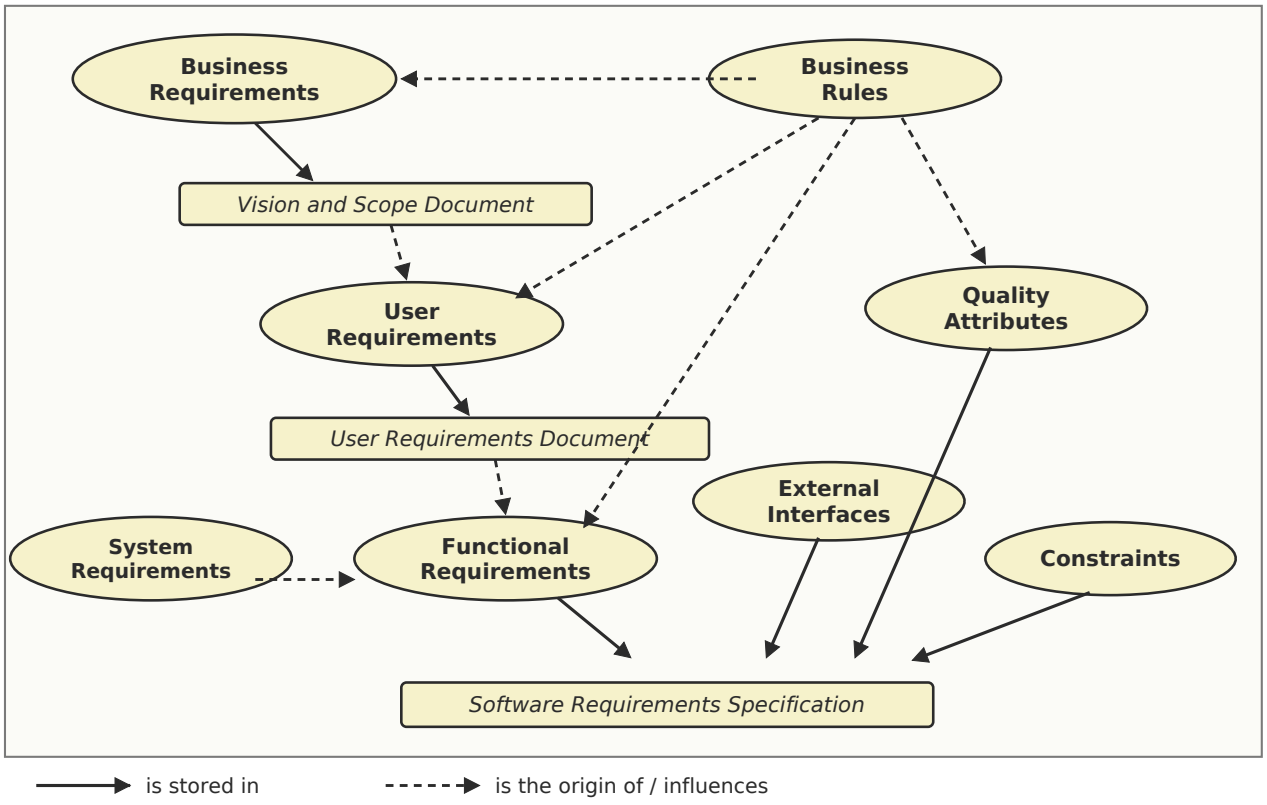
Feature: **Live Order Tracking**. Its user requirements: see the rider on a map; see an ETA; call the rider. Each implies functional requirements — e.g., "the system shall refresh rider GPS position every 10 seconds", "shall compute ETA from live traffic", "shall connect calls through a masked number."

MEMORY HOOK **Feature ⊃ user requirements ⊃ functional requirements** — a tree, one-to-many at each step (picture the browser's Bookmarks branch).

1.8 How all the requirement types connect

CH-1 · SLIDE 14

The slides summarize everything so far in one map. Two arrow types carry the meaning: **solid = "is stored in"** (which document holds it) and **dashed = "is the origin of / influences"**.



Business requirements are stored in the Vision & Scope document; user requirements in the User Requirements Document; functional requirements, quality attributes, external interfaces and constraints in the SRS. Business rules influence business & user requirements, quality attributes and functional requirements; system requirements are an origin of functional requirements.

Read it as three document shelves: **Vision & Scope** (why), **User Requirements Document** (what users do), **SRS** (everything developers need: functional requirements + quality attributes + external interfaces + constraints). **Business rules float outside** all documents — they only influence.

MEMORY HOOK Solid arrow = **filed in**; dashed arrow = **gives birth to**. Business rules never live in a shelf — they whisper into everything.

1.9 Product vs. project requirements

CH-1 · SLIDES 15-16

Not everything a project needs is part of the *product*:

Kind	Meaning
Product requirements	Describe **properties of the software system to be built**. Housed in the **SRS** — which should **not** include design or implementation details (other than known constraints), project plans, test plans, or similar. Separating them lets requirements development focus on **what the team intends to build**.
Project requirements	Other expectations and deliverables that are **not part of the software** but are necessary for successful completion of the project as a whole (slide example: maintaining the schedule deadline of a task).

The seven categories of project requirements (slide 16) — memorize with the **7 G's**:

- **Gear** — physical resources: workstations, special hardware devices, testing labs, testing tools and equipment, team rooms, videoconferencing equipment.
- **Grow** — staff training; user documentation including training materials, tutorials, reference manuals, release notes.
- **Ground** — infrastructure changes needed in the operating environment.
- **Go-live** — requirements and procedures for releasing the product, installing it in the operating environment, configuring it, and testing the installation.
- **Go-to-market** — beta testing, manufacturing, packaging, marketing, and distribution requirements.
- **Guarantees** — customer service-level agreements.
- **Guard** — legal protection (patents, trademarks, copyrights) for intellectual property related to the software.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Product: everything the QuickBite apps do. Project: 30 test phones (Gear), rider training videos (Grow), new staging servers (Ground), the Play Store release checklist (Go-live), the launch campaign (Go-to-market), the "we answer restaurant tickets within 4 hours" SLA (Guarantees), trademarking the QuickBite logo (Guard).

MEMORY HOOK Product = **what the software IS**; project = **what it takes to ship it**. Project needs come in **7 G's: Gear, Grow, Ground, Go-live, Go-to-market, Guarantees, Guard**.

QUICK RECALL — PART 1

- 10 classic problems = Foggy start / Blind build / Change chaos / Hollow finish.
- Requirement = value to a stakeholder; 2 views (external/internal); 3 tenses.
- Levels: Business (why) → User (what user does) → Functional (what devs build) — alignment = success.
- System = subsystems + hardware + **people & processes** + interfaces.
- NFR = -ilities (how well); they conflict with each other easily.
- Business rules exist outside software; drive functionality, constraints, quality attributes; traceable origins.
- Feature ⊃ user requirements ⊃ functional requirements.
- Docs: Vision & Scope ← business; URD ← user; SRS ← functional + quality + interfaces + constraints.
- Product vs project: SRS holds product only; project needs = the 7 G's.

PART 2

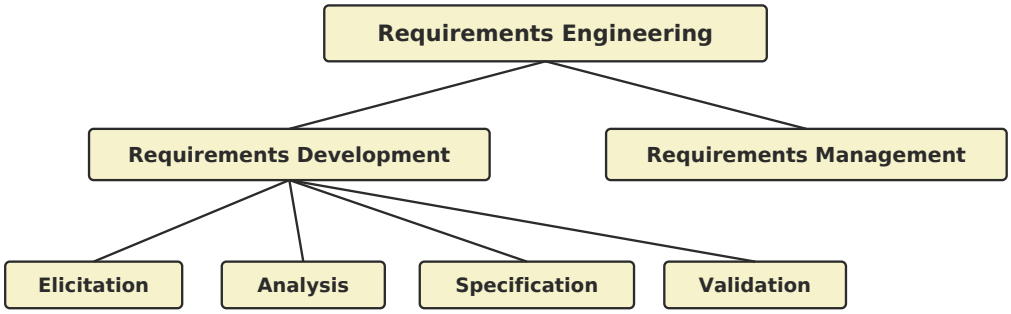
The Requirements Engineering Process

Chapter 1 · Development (elicitation, analysis, specification, validation) + management — and why it all matters

2.1 The big picture: development + management

CH-1 · SLIDE 17

Requirements engineering splits into two subdisciplines. **Requirements development** creates good requirements through four activities; **requirements management** keeps them under control once agreed. Draw this tree from memory:



Subdisciplines of software requirements engineering: development (elicitation, analysis, specification, validation) and management.

MEMORY HOOK

RE = Development + Management. Development = **E-A-S-V** (Elicit, Analyze, Specify, Validate — "Every Analyst Speaks Volumes"). Management = guarding the agreed set.

2.2 Requirements development: ELICITATION

CH-1 · SLIDE 18

Elicitation is the **discovery** step — four activities, memorable as **WHO / WHAT / WHERE / WITH**:

- **WHO** — identify the product's **expected user classes and other stakeholders**.
- **WHAT** — understand **user tasks and goals** and the **business objectives** those tasks align with.
- **WHERE** — learn about the **environment** in which the new product will be used.
- **WITH** — **work with individuals representing each user class** to understand their functionality needs and quality expectations.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

WHO: diners, riders, restaurant owners, ops admins. WHAT: order food fast; deliver efficiently; manage menus — aligned to the COO's cost goal. WHERE: noisy kitchens, riders in traffic on budget phones with patchy data. WITH: sit weekly with 5 diners, 3 riders, 2 restaurant owners.

MEMORY HOOK

Elicitation = **WHO, WHAT, WHERE, WITH** — four W's of discovery.

2.3 Requirements development: ANALYSIS

CH-1 · SLIDE 19

Analysis turns raw input into structured understanding — five activities, mnemonic **“McDAN”** (Model, Classify, Decompose, Allocate, Negotiate):

- **Model** the application environment.
- **Classify** — analyse the information received from users to distinguish task goals into functional requirements, quality expectations, business rules, suggested solutions, and more.
- **Decompose** high-level requirements into an appropriate level of detail.
- **Allocate** requirements to the software components defined in the system architecture.
- **Negotiate** requirements priority and their implementation priorities.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Model the order flow diner→restaurant→rider. Classify: “app must be fast” = quality expectation; “Reorder button” = functional; “hygiene license required” = business rule; “use Firebase” = a suggested solution, not a requirement. Decompose “manage menu” into add/edit/disable item. Allocate: menu editing → restaurant tablet; tracking → customer app. Negotiate: live tracking ships before coupons.

MEMORY HOOK Analysis = **McDAN**: Model · Classify · Decompose · Allocate · Negotiate.

2.4 Requirements development: SPECIFICATION

CH-1 · SLIDE 20

Specification is **writing it down well**. The business analyst documents requirements in a **software requirements specification (SRS)**:

- **Transcribe** the collected user needs into **written requirements and diagrams** suitable for comprehension, review, and use by their intended audiences.
- The SRS describes **as fully as necessary the expected behaviour** of the software system.
- The SRS is **used by** development, testing, quality assurance, project management, and related project functions.
- Adopt **requirement document templates** — a well-defined standard of writing requirements.
- **Identify requirement origins** (where each came from).
- **Uniquely label** each requirement — essential for **cross-cutting** requirements referenced in many places.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

QuickBite’s SRS uses a fixed template; requirement FR-ORD-07 (“rebuild cart ≤2 s”) records its origin — “COO interview, 12 Mar” — and its unique label lets the payment section and the offline section both reference it without copy-pasting.

MEMORY HOOK Specification = SRS + **TOL**: **T**emplates, **O**rigins, **L**abels.

2.5 Requirements development: VALIDATION

CH-1 · SLIDE 21

Validation asks: *did we write the right requirements, well?* Four moves — **Review it, Prune it, Prove it, Preview it**:

- **Review it** — review the documented requirements (SRS) to correct problems **before the development group accepts them**; check they are **feasible, consistent, complete**. **Consistency** = no requirements contradict each other; **completeness** = no needed services or constraints have been missed.
- **Prune it** — also **discard unnecessary requirements**.
- **Prove it** — develop **acceptance tests and criteria** to confirm a product based on these requirements would meet customer needs and achieve the business objectives.
- **Preview it** — **simulate the requirements** (e.g., wireframing) to find errors early.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

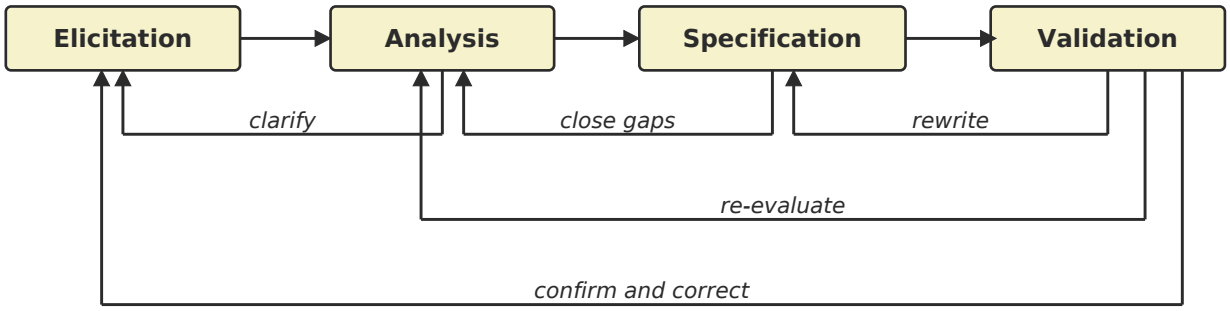
Review finds a contradiction: §3 says “all payments online” while §7 describes cash-on-delivery — inconsistent. Prune: drop the “social feed” nobody asked for. Prove: “Given a past order, when I tap Reorder, the cart rebuilds in ≤2 s.” Preview: clickable wireframes with 5 diners expose a confusing checkout before a line of code is written.

MEMORY HOOK Validation = **Review it · Prune it · Prove it · Preview it**. Consistent = no contradictions; Complete = nothing missing.

2.6 The framework is iterative, not linear

CH-1 · SLIDES 22-23, 26

E-A-S-V is **not a one-way street**. Each later stage sends work backwards — learn each back-arrow with its label:

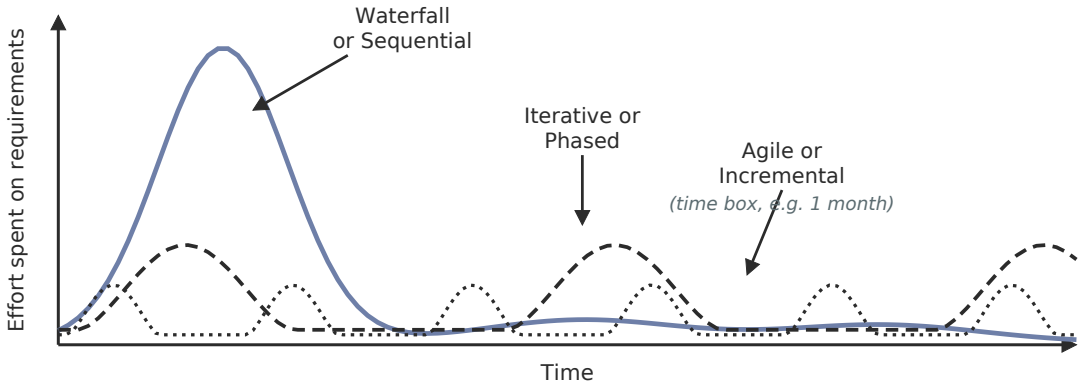


Requirements development is an iterative process: analysis sends questions back to elicitation (clarify); specification exposes missing pieces (close gaps); validation forces rewrites of the spec (rewrite), re-evaluation of the analysis (re-evaluate), and confirmation/correction with the sources (confirm and correct).

The slides then give a **representative requirements development process** — 17 numbered steps: seven done **once up front**, ten repeated **every iteration**:

ONCE, UP FRONT (steps 1–7)	EVERY ITERATION (steps 8–17) → repeat for iteration 2, 3 … N
1. Define business requirements 2. Identify user classes 3. Identify user representatives 4. Identify requirements decision makers 5. Plan elicitation 6. Identify user requirements 7. Prioritize user requirements	8. Flesh out (expand) user requirements 9. Derive functional requirements 10. Model the requirements 11. Specify nonfunctional requirements 12. Review requirements 13. Develop prototypes 14. Develop or evolve architecture 15. Allocate requirements to components 16. Develop tests from requirements 17. Validate user, functional and nonfunctional requirements, analysis models, and prototypes

How much requirements effort, and when? It depends on the life cycle:



Distribution of requirements development effort over time for different life cycles: waterfall/sequential = one big up-front hump; iterative/phased = medium humps per phase; agile/incremental = small frequent humps, one per time box (the slide notes a time box of one month).

MEMORY HOOK Back-arrows: **clarify** (A→E), **close gaps** (S→A), **rewrite** (V→S), **re-evaluate** (V→A), **confirm & correct** (V→E). Steps: **7 once + 10 per iteration = 17**. Effort curves: waterfall one mountain, agile many hills.

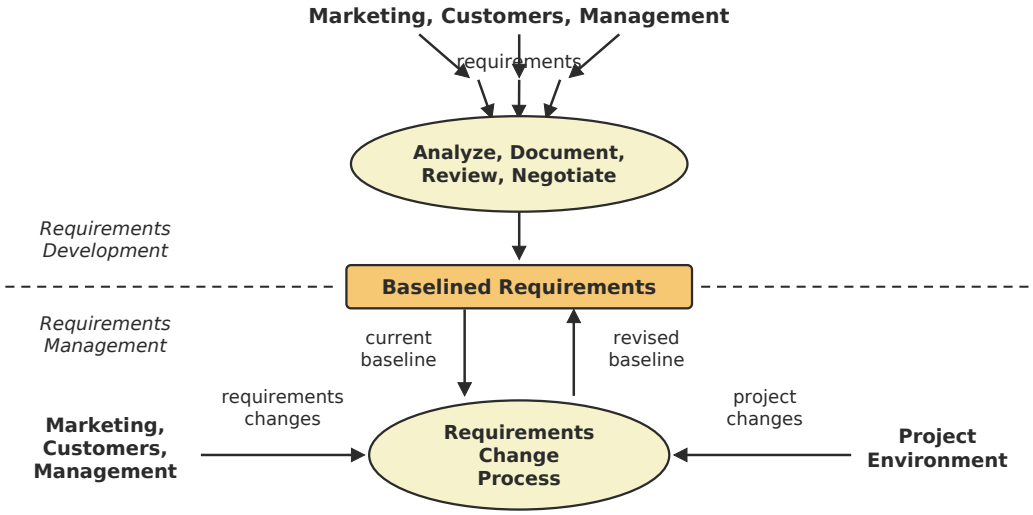
2.7 Requirements MANAGEMENT

CH-1 · SLIDES 24–25

Once requirements are agreed, management keeps them trustworthy. Seven activities — chunk them as **Control, Chronicle, Connect**:

- **Control** — establish a **requirements change control process**; perform **impact analysis** on requirements changes.
- **Chronicle** — **track the status** of each requirement; **track requirements issues**; maintain a **history of requirements changes**.
- **Connect** — maintain a **requirements traceability matrix**; define the **relationships and dependencies** between requirements; **trace individual requirements to their corresponding designs, source code, and tests**.

Where development ends and management begins — the **baseline boundary**:



Marketing, customers and management supply requirements; the team analyzes, documents, reviews and negotiates them (development) into baselined requirements. Below the line (management), the requirements change process takes the current baseline plus requirement changes (from marketing/customers/management) and project changes (from the project environment) and produces a revised baseline.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Post-baseline, marketing requests “scheduled orders.” Change control: the request is logged (never lost), impact analysis says +2 weeks, the decision makers trade it against the coupon feature, status moves proposed → approved → implemented → verified, and FR-SCH-01 is traced to its design, code module, and test case TC-31.

MEMORY HOOK Management = **3 C’s**: **C**ontrol change · **C**hronicle status/history · **C**onnect via traceability. The **baseline** is the border crossing between development and management.

2.8 Why requirements matter — Brooks’ warning

CH-1 · SLIDE 27

The slide quotes Frederick Brooks’ famous argument that deciding precisely what to build is “the hardest single part of building a software system.” No other part of the conceptual work is as difficult as establishing the detailed technical requirements — including all the interfaces to people, to machines, and to other software systems; no other part so cripples the resulting system if done wrong; and no other part is more difficult to fix later.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Getting “is QuickBite delivery-only, or also pickup?” wrong poisons the database schema, the rider app, the restaurant tablet, pricing — everything. Fixing that decision after launch costs more than any coding bug.

MEMORY HOOK Brooks: requirements are the **hardest** part, the most **crippling** if wrong, the most **expensive to fix later** — three superlatives.

2.9 Reasons behind bad requirements

CH-1 · SLIDE 28

Six causes — mnemonic **“CIAO, GI!”** (wave goodbye to your project):

- **C**reeping user requirements — gradually increasing, changing scope.
- **I**nsufficient user involvement.
- **A**mbiguous requirements — slide example: “respond to a request *as soon as possible*” (as soon as what, exactly?).
- **O**verlooked stakeholders.
- **G**old plating — extra functionality beyond the specification.
- **I**naccurate planning.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Gold plating: a developer adds animated confetti on checkout that nobody requested — effort spent, risk added, zero required value. Ambiguity: “the app should be fast” became “≤3 s on 3G” only after validation caught it.

MEMORY HOOK Bad requirements? Say **CIAO, GI**: Creeping, Insufficient involvement, Ambiguous, Overlooked stakeholders, Gold plating, Inaccurate planning.

2.10 Benefits of a high-quality requirements process

CH-1 · SLIDE 29

Ten benefits — remember them as **7 FEWER + 3 WINS**:

Group	The ten benefits
7 FEWER / LOWER / REDUCED	Fewer **defects** in requirements and in the delivered product · reduced development **rework** · fewer **unnecessary and unused features** · lower **enhancement costs** · fewer **miscommunications** · reduced **scope creep** · reduced **project chaos**.
3 WINS	**Faster** development and delivery · **higher** customer and team member satisfaction · products that **do what they’re supposed to do**.

MEMORY HOOK **7 fewer + 3 wins = 10 benefits.** Less waste (defects, rework, unused features, cost, miscommunication, creep, chaos); more speed, satisfaction, and a product that does its job.

QUICK RECALL — PART 2

- RE tree: Development (E-A-S-V) + Management — draw it.
- Elicitation = WHO, WHAT, WHERE, WITH.
- Analysis = McDAN (Model, Classify, Decompose, Allocate, Negotiate).
- Specification = SRS + TOL (Templates, Origins, Labels).
- Validation = Review it, Prune it, Prove it (acceptance tests), Preview it (wireframes); consistent + complete + feasible.
- Iteration back-arrows: clarify, close gaps, rewrite, re-evaluate, confirm & correct.
- Process: 7 up-front steps + 10 per-iteration steps (repeat for iteration 2, 3, N).
- Management = Control (change process, impact analysis), Chronicle (status, issues, history), Connect (traceability matrix, dependencies, trace to design/code/tests).
- Baseline = boundary between development and management.
- Effort curves: waterfall one big early hump; iterative medium humps; agile small monthly humps.
- Brooks: hardest part, cripples if wrong, hardest to fix later.
- Bad requirements = CIAO, GI. Benefits = 7 fewer + 3 wins.

PART 3

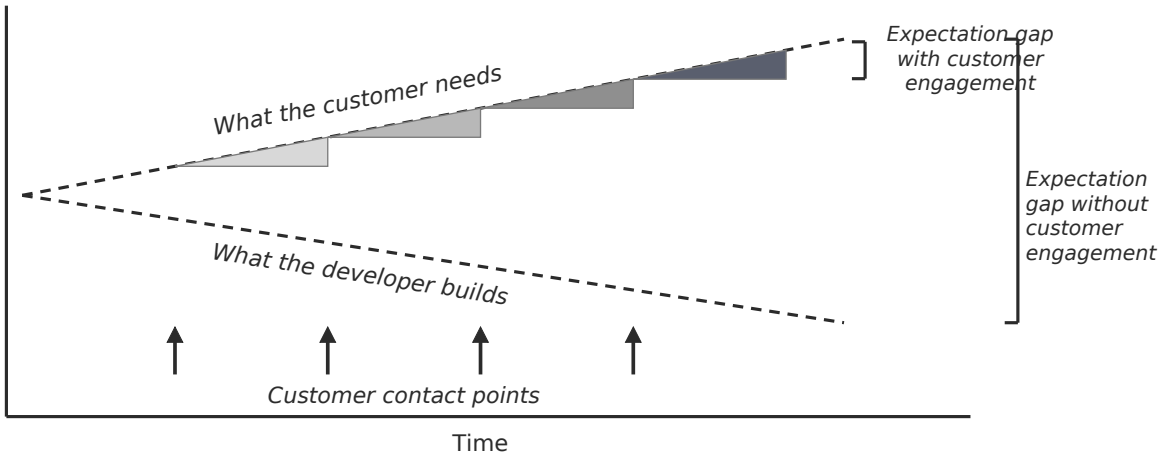
Stakeholders, Customers & the Partnership

Chapter 2 · Who the players are, what each side owes the other, and how agreement is reached

3.1 The expectation gap

CH-2 · SLIDE 2

Over time, **what the customer needs** drifts upward while **what the developer builds** (without feedback) drifts away from it. The distance between the two lines is the **expectation gap**. Every **customer contact point** — a demo, a review, a conversation — snaps the two lines back together, so the gap only grows a little between contacts. Without engagement, it grows unchecked.



Frequent customer engagement reduces the expectation gap: with contact points, only small (darkening) triangles of mismatch accumulate; without them, the full distance between "needs" and "builds" opens up.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

The COO imagined a premium dark UI with a live animated map. The team coded for four months with no demos and unveiled a plain grey list. Four sprint demos would have caught the drift at triangle #1 — the cost of realignment grows with every skipped contact point.

MEMORY HOOK Two diverging dashed lines + shrinking triangles = **engage often, gap stays small**. More contact points ⇒ smaller expectation gap.

3.2 Stakeholders: the full cast

CH-2 · SLIDE 3

DEFINITION — Stakeholder

A **person, group, or organization** that is **actively involved in** a project, is **affected by its process or outcome**, or **can influence** its process or outcome. Stakeholders can be **internal or external** to the project team and to the developing organization.

The slides map potential stakeholders as three nested zones — recreated here. You don't need every role by heart; know the **three zones**, a few examples per zone, and that the lists are long on purpose (stakeholders are easy to overlook — problem cause "O" in CIAO, GI!):

OUTSIDE THE DEVELOPING ORGANIZATION

Direct user · indirect user · acquirer · procurement staff · legal staff · contractor · subcontractor · business management · contracting officer · government agency · subject matter expert · program manager · beta tester · general public · consultant · compliance auditor · certifier · regulatory body · software supplier · materials supplier · venture capitalist

DEVELOPING ORGANIZATION

Development manager · marketing · operational support staff · legal staff · information architect · company owner · sales staff · installer · maintainer · program manager · usability expert · subject matter expert · executive sponsor · project management office · manufacturing · training staff · portfolio architect · infrastructure support staff

PROJECT TEAM

Project manager · business analyst · application architect · designer · developer · product owner · data modeler · process analyst · tester · product manager · quality assurance staff · documentation writer · database administrator · hardware engineer · infrastructure analyst · business solutions architect

Recreation of the slide's Figure 2-2: potential stakeholders within the project team, within the developing organization, and outside the organization.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Project team: QuickBite's PM, BA, developers, testers, designer. Developing organization: the COO (executive sponsor), marketing, support, legal. Outside: diners and riders (direct users), the finance manager (indirect user), the payment-gateway company (software supplier), the food-safety authority (regulatory body), investors (venture capitalists).

MEMORY HOOK Stakeholder = **involved, affected, or influential** — any one of the three qualifies. Picture **three nested rings**: team ⊂ organization ⊂ the outside world.

3.3 Who is the customer? (and who are users?)

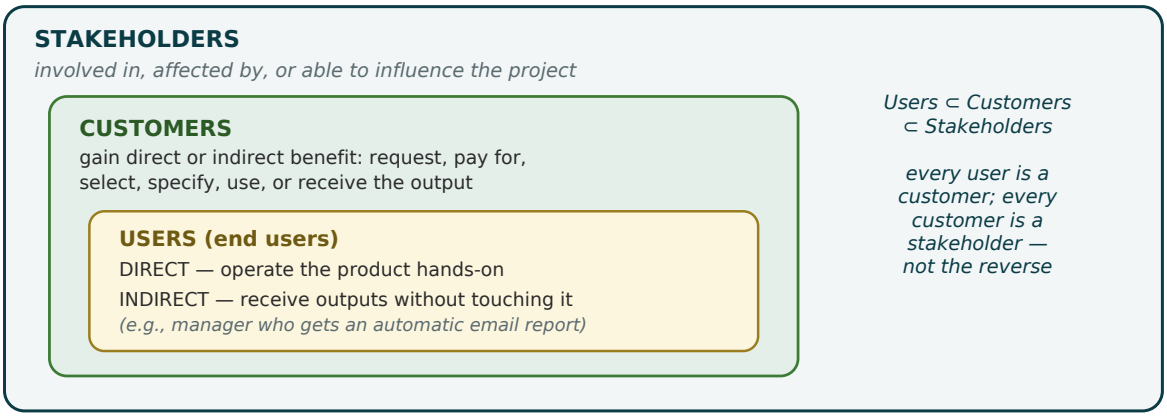
CH-2 · SLIDE 4

DEFINITION — Customer

A **subset of stakeholders**: a person or organization that **derives direct or indirect benefit** from a product. Customers **request, pay for, select, specify, use, or receive the output** generated by a software product.

Inside the customer circle sit the **users** (end users), and they split in two:

- **Direct users** — operate the product **hands-on**.
- **Indirect users** — **receive output** from the product **without touching it themselves**. Slide example: a **warehouse manager who automatically receives a daily activity report by email**.
- Expect users to describe **tasks** they must perform, the **outputs** they need, and the **quality** they expect — that's the information to elicit from them.



Users ⊂ customers ⊂ stakeholders: every user is a customer and every customer is a stakeholder — but not the reverse.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

The COO pays for QuickBite — an acquiring customer who may never place an order. Diners, riders and restaurant staff are direct users. The finance manager who just receives the automatic daily sales email is an indirect user — a customer all the same, with real requirements about that report.

MEMORY HOOK **Users ⊂ Customers ⊂ Stakeholders.** Direct = hands ON it; indirect = output FROM it (the emailed-report manager).

3.4 The customer's nine RIGHTS

CH-2 · SLIDE 5

Rights = expectations that customers can **legitimately hold** about their interactions with BAs and developers during the project's requirements engineering activities — so every right is **a responsibility (an obligation) of the business analyst** to honour. Mnemonic: **BLUR CRIES** (ignore customers' rights and their cries blur together):

- **B**usiness — expect BAs to **learn your business and your objectives**.
- **L**anguage — expect BAs to **speak your language**.
- **U**sability — **describe characteristics that make the product easy to use**.
- **R**euse — **hear about ways to adjust requirements to accelerate development through reuse**.
- **C**hange — **change your requirements**.
- **R**espect — an **atmosphere of mutual respect**.
- **I**deas — **hear ideas and alternatives** for your requirements and their solutions.
- **E**xplanations — **receive explanations of requirements practices and deliverables**.
- **S**ystem — **receive a system that meets your functional needs and quality expectations**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Restaurant owners exercise their rights: the BA talks in *menus, prep times, commissions* — not APIs (Language); they hear the alternative "a cheap Android tablet instead of custom hardware" (Ideas/Reuse); and mid-way they change the requirement from printed receipts to digital ones (Change).

MEMORY HOOK Rights = **BLUR CRIES**: Business, Language, Usability, Reuse, Change, Respect, Ideas, Explanations, System. Nine rights = nine BA obligations.

3.5 The customer's nine RESPONSIBILITIES

CH-2 · SLIDE 6

The partnership runs both ways. **Responsibilities** = what the customer **owes BAs and developers during the requirements process** — so each one is **a right of the business analyst**. Mnemonic sentence: **“Every Tuesday Students Deliver Facts, Prioritize Revision, Ace Papers.”**

- **E**ducate — educate BAs and developers **about your business**.
- **T**ime — **dedicate the time** it takes to provide and clarify requirements.
- **S**pecific — be **specific and precise** when providing input about requirements.
- **D**ecisions — **make timely decisions** about requirements when asked (e.g., on user-friendliness or security trade-offs).
- **F**easibility — **respect a developer’s assessment of cost and feasibility**.
- **P**riorities — collaborate in **setting realistic priorities** for requirements.
- **R**eview — **review requirements documents and evaluate prototypes** (after each iteration).
- **A**cceptance — **establish acceptance criteria** and **promptly communicate changes** to the requirements.
- **P**rocess — **respect the requirements development process** the team follows.

RIGHTS	RESPONSIBILITIES
What you **RECEIVE** from the partnership — and the **BA must provide**. Think BLUR CRIES.	What you **GIVE** to the partnership — and the **BA may demand**. Think Every Tuesday Students…

EXAM TRAP

Classic exam trap: the direction. **Customer rights are the BA’s responsibilities; customer responsibilities are the BA’s rights.** “Change your requirements” is a customer *right*; “promptly communicate changes” is a customer *responsibility* — same event, two sides.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Rights in action vs. responsibilities in action: the owners may change the receipt requirement (right) — but must tell the BA within days, not at launch (responsibility); the COO may hear alternatives (right) — but must respect the estimate that real-time GPS on 2G phones is infeasible (responsibility).

MEMORY HOOK Responsibilities = **“Every Tuesday Students Deliver Facts, Prioritize Revision, Ace Papers”** → Educate, Time, Specific, Decisions, Feasibility, Priorities, Review, Acceptance, Process.

3.6 Making decisions & reaching agreement

CH-2 · SLIDE 7

Requirements conflicts are normal — someone must decide. The slides give **four decision-making methods**, in widening circles of inclusion:

- **The leader decides** — with or without discussion.
- **Majority vote** — most win, some lose.
- **Unanimous vote** — all must agree.
- **Consensus** — discuss and negotiate until **everyone can live with the decision and commits to it**.

And **who must agree** on the requirements before they can be baselined? Four groups, each checking something different — mnemonic **“Can Devs Test Money?”**:

- **C**ustomers — agree the requirements **address their needs**.
- **D**evelopers — agree the requirements are **understood and feasible**.
- **T**esters — agree the requirements are **verifiable**.

- **M**anagement — agree the requirements will **achieve the business objectives**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

For “COD limit: 3 or 5?” the product owner, BA and lead developer talk it out to consensus; if talks stall, the product owner (leader) decides. Before freezing v1.0: diner reps confirm needs, developers confirm feasibility, QA confirms every “shall” is testable, the COO confirms the cost goal is still met.

MEMORY HOOK Decisions widen: **One → Most → All → Live-with-it (consensus)**. Agreement: **Can Devs Test Money?** — needs, feasible, verifiable, objectives.

3.7 The requirements baseline — and the confidence it buys

CH-2 · SLIDE 8

DEFINITION — Requirements baseline

A set of requirements that has been **reviewed and agreed upon**, and serves as the **basis for further development**.

Why bother baselining? Because it hands **five groups** five different kinds of confidence:

- **Customer management / marketing** — confidence the **scope won’t explode**, because customers share in managing scope decisions.
- **User representatives** — confidence the team will **deliver the right solution**, even though not every requirement is known up front.
- **Development management** — confidence in a **business partner who keeps the project focused on its objectives** and helps balance schedule, cost, functionality, and quality.
- **Business analysts & project managers** — confidence they can **manage change with minimal chaos**.
- **QA / testers** — confidence they can **develop test scripts** and be **fully prepared**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

v1.0 is baselined after the four-group review. Now marketing trusts scope, diner reps trust the direction, dev management trusts the partnership, the BA & PM handle “scheduled orders” through change control instead of chaos, and QA starts writing test scripts the same afternoon.

MEMORY HOOK Baseline = **reviewed + agreed + the base to build on**. It buys confidence for **5 groups**: customer mgmt, user reps, dev mgmt, BA & PM, QA/testers.

QUICK RECALL — PART 3

- Expectation gap = needs vs. builds; contact points keep the triangles small.
- Stakeholder = involved / affected / influential; three nested zones (team ⊂ org ⊂ outside).
- Users ⊂ customers ⊂ stakeholders; direct = hands-on, indirect = receives outputs (emailed report).
- 9 rights = BLUR CRIES (BA’s obligations).
- 9 responsibilities = Every Tuesday Students Deliver Facts, Prioritize Revision, Ace Papers (BA’s rights).
- Decisions: leader → majority → unanimous → consensus (live-with-it + commit).
- Agreement: Can Devs Test Money? (needs / feasible / verifiable / objectives).
- Baseline = reviewed & agreed basis; buys confidence for 5 groups.

PART 4

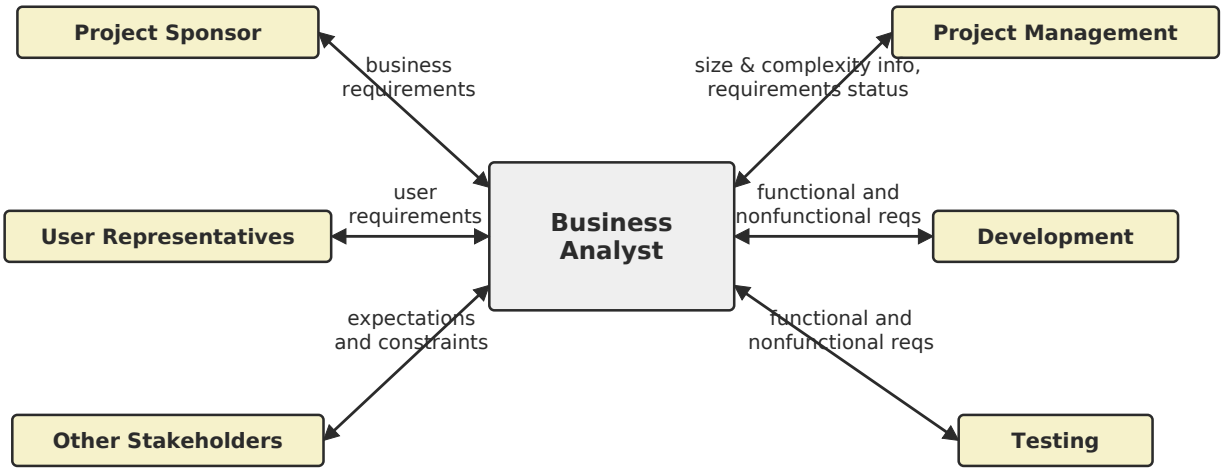
The Business Analyst

Chapter 2 · The bridge role: tasks, skills, knowledge, and how people become BAs

4.1 The BA role: a communication bridge

CH-2 · SLIDE 9

The **business analyst** (also called requirements analyst, requirements engineer, or simply analyst) sits between the customer world and the delivery world and **translates in both directions**:



The BA bridges communication: business requirements flow with the project sponsor; user requirements with user representatives; expectations and constraints with other stakeholders. Toward delivery: size & complexity information and requirements status with project management; functional and nonfunctional requirements with development and with testing.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

QuickBite's BA turns the COO's "cut call-center cost 40%" into user stories developers can build; carries the regulator's hygiene-license constraint into the SRS; reports "60% of v1 requirements verified, tracking feature is high-complexity" to the PM; and hands QA testable functional and nonfunctional requirements.

MEMORY HOOK BA = **bridge with six lanes**: sponsor, user reps, other stakeholders on one side — PM, development, testing on the other. Every arrow is **two-way**.

4.2 The BA's ten tasks

CH-2 · SLIDE 10

Ten principal activities — mnemonic: **"Don't Panic! I Eat And Drink Coffee Very Promptly, Mate."**

- **D**on't → **Define business requirements**.
- **P**anic → **Plan the requirements approach** for the project.
- **I** → **Identify project stakeholders and user classes**.
- **E**at → **Elicit requirements**.
- **A**nd → **Analyze requirements**.

- **D**rink → **Document requirements**.
- **C**offee → **Communicate requirements**.
- **V**ery → **Lead requirements validation**.
- **P**romptly → **Facilitate requirements prioritization**.
- **M**ate → **Manage requirements**.

Notice the skeleton inside: tasks 4–8 are exactly the development activities you already know (elicit → analyze → document/specify → communicate → validate), book-ended by set-up work (define, plan, identify) and follow-through (prioritize, manage).

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

One release, ten tasks: the BA defined the COO's goal, planned the interview calendar, identified diner/rider/restaurant/admin user classes, elicited weekly, analyzed inputs into requirements types, documented the SRS, communicated it through demos, led the v1.0 validation review, facilitated the "tracking before coupons" priority call, and managed the scheduled-orders change afterwards.

MEMORY HOOK **Don't Panic! I Eat And Drink Coffee Very Promptly, Mate** = Define, Plan, Identify, Elicit, Analyze, Document, Communicate, Validate, Prioritize, Manage — exactly 10.

4.3 Essential analyst SKILLS (all 14)

CH-2 · SLIDE 11

Fourteen skills — don't memorize a 14-letter monster; chunk them **6 + 5 + 3**:

Chunk	The skills
6 PEOPLE skills *talk & lead*	**Listening** · **interviewing and questioning** · **communication** · **interpersonal** · **facilitation** (and simplification) · **leadership**.
5 THINKING skills *process & invent*	**Analytical** · **systems thinking** · **thinking on your feet** · **learning** · **creativity**.
3 CRAFT skills *see & shape*	**Observational** · **organizational** · **modeling**.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

Thinking on your feet: a restaurant owner derails the interview — the BA pivots the questions live. Modeling: sketching the diner→restaurant→rider order flow on a whiteboard. Facilitation: turning a shouting match about commissions into a ranked priority list.

MEMORY HOOK Skills = **6 talk, 5 think, 3 craft** — 6+5+3 = 14.

4.4 Essential analyst KNOWLEDGE

CH-2 · SLIDE 12

Skills are not enough — the slide adds two knowledge demands:

- A BA needs **breadth of knowledge that comes from experience** — seasoning across projects, domains, and situations.
- Command of **contemporary requirements engineering practices** — and the ability to **apply them across different software development life cycles** (waterfall, iterative, agile…).

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

QuickBite runs monthly agile time boxes, but the payment-gateway integration is contract-driven and waterfall-ish. The same BA must run lightweight story refinement for one and a formal SRS review for the other — same practices toolbox, different life cycles.

MEMORY HOOK Knowledge = **experience breadth + modern RE practices, portable across SDLCs**.

4.5 The making of a business analyst — four paths

CH-2 · SLIDE 13

BAs are made, not born — people typically arrive from one of **four backgrounds**, each bringing a natural strength:

- **The former USER** — knows the business tasks and pains first-hand.
- **The former DEVELOPER or TESTER** — knows what is feasible and what is verifiable.
- **The former (or concurrent) PROJECT MANAGER** — knows how to balance scope, schedule, cost, and people.
- **The SUBJECT MATTER (domain) EXPERT** — knows the domain's rules and vocabulary in depth.

RUNNING EXAMPLE — QUICKBITE (food-delivery app)

QuickBite's BA is a former restaurant manager — a domain expert who already speaks "prep time" and "rush hour," which is exactly why elicitation with owners goes fast.

MEMORY HOOK Four roads to the BA chair: **User, Developer/Tester, Project Manager, Domain expert** — every seat at the project table can slide into it.

QUICK RECALL — PART 4

- BA = two-way bridge: sponsor / user reps / other stakeholders ↔ BA ↔ PM / development / testing.
- 10 tasks = Don't Panic! I Eat And Drink Coffee Very Promptly, Mate.
- 14 skills = 6 people + 5 thinking + 3 craft.
- Knowledge = experience breadth + contemporary RE practices across SDLCs.
- 4 paths to BA: user, developer/tester, project manager, domain expert.

PART 5

Rapid Revision

The whole guide compressed to trigger lines — then test yourself with the question bank

5.1 One-look cheat sheet

EVERYTHING · COMPRESSED

Definitions in one breath

Term	Trigger line
Requirement	Property giving a stakeholder value · 2 views (external behaviour / internal traits) · 3 tenses (past, present, future: near-term high priority, hypothetical low).
Business req	WHY — the org's benefit. Sources: funding sponsor, acquiring customer, users' manager, marketing, product visionary (sources may conflict). Lives in Vision & Scope.
User req	WHAT users do — goals/tasks giving value + attributes for satisfaction. Use cases & user stories, from user reps. Lives in the URD.
Functional req	WHAT developers build — behaviours under specific conditions; "shall" statements. Lives in the SRS.
Constraint	Restriction on developer choices (open-source tools + Linux; Android 8+).
System req	Multi-subsystem product: software + hardware + **people & processes**; derive per-subsystem functionality + interfaces (cashier workstation).
NFR / quality attribute	HOW WELL — the -ilities (performance, safety, availability, portability…); conflicts among them are fairly likely; includes external-interface qualities (interoperability).
Business rule	Policy / regulation / standard / algorithm; exists **outside** the software; drives compliance functionality, constraints, quality attributes; origins are traceable. Inverse requirement: "no red in the webpage."
Feature	Logically related capabilities with value ⊃ user reqs ⊃ functional reqs (browser → Bookmarks → Edit Bookmarks → 4 FRs).
SRS	Expected behaviour, as fully as necessary; used by dev, testing, QA, PM; excludes design/implementation details (except known constraints), project & test plans.
Stakeholder	Involved in, affected by, or able to influence — internal or external; three zones: team ⊂ developing org ⊂ outside.
Customer / user	Customer ⊂ stakeholders: request, pay, select, specify, use, or receive output. Users ⊂ customers: direct = hands-on; indirect = receives outputs (daily emailed report).
Expectation gap	Needs-line vs builds-line drift; contact points reset it — frequent engagement keeps triangles small.
Baseline	Reviewed + agreed set; basis for further development; boundary between requirements development and management.
Business analyst	Two-way bridge: sponsor / user reps / stakeholders ↔ BA ↔ PM / development / testing.

Every list, its count, its mnemonic

List	Count & mnemonic
10 classic problems	Foggy start (3) → Blind build (2) → Change chaos (3) → Hollow finish (2).
RE tree	Development (E-A-S-V) + Management.
Elicitation · 4	WHO, WHAT, WHERE, WITH.
Analysis · 5	McDAN: Model, Classify, Decompose, Allocate, Negotiate.

List	Count & mnemonic
Specification	SRS + TOL: Templates, Origins, Labels (unique labels serve cross-cutting reqs).
Validation · 4	Review it (feasible/consistent/complete), Prune it, Prove it (acceptance tests & criteria), Preview it (simulate/wireframe).
Iteration back-arrows · 5	clarify (A→E), close gaps (S→A), rewrite (V→S), re-evaluate (V→A), confirm & correct (V→E).
Process steps · 17	7 once up front + 10 every iteration; repeat for iteration 2, 3 … N.
Management · 7 acts	3 C's — Control (change process, impact analysis), Chronicle (status, issues, history), Connect (traceability matrix, dependencies, trace to design/code/tests).
Effort curves	Waterfall: one big early hump · iterative: medium humps · agile: small monthly humps (time box ≈ 1 month).
Brooks	Hardest single part · cripples if wrong · hardest to fix later.
Bad requirements · 6	CIAO, GI! — Creeping, Insufficient involvement, Ambiguous ("as soon as possible"), Overlooked stakeholders, Gold plating, Inaccurate planning.
Benefits · 10	7 FEWER (defects, rework, unused features, enhancement cost, miscommunication, scope creep, chaos) + 3 WINS (faster delivery, satisfaction, does what it should).
Project reqs · 7	7 G's: Gear, Grow, Ground, Go-live, Go-to-market, Guarantees, Guard.
Customer rights · 9	BLUR CRIES: Business, Language, Usability, Reuse, Change, Respect, Ideas, Explanations, System. (= BA's obligations)
Customer responsibilities · 9	Every Tuesday Students Deliver Facts, Prioritize Revision, Ace Papers: Educate, Time, Specific, Decisions, Feasibility, Priorities, Review, Acceptance, Process. (= BA's rights)
Decision methods · 4	Leader → Majority → Unanimous → Consensus (live with it + commit).
Agreement groups · 4	Can Devs Test Money? — Customers: needs · Developers: understood & feasible · Testers: verifiable · Management: business objectives.
Baseline confidence · 5	Customer mgmt/marketing (scope) · user reps (right solution) · dev mgmt (focused partner) · BA & PM (change, minimal chaos) · QA/testers (test scripts, prepared).
BA tasks · 10	Don't Panic! I Eat And Drink Coffee Very Promptly, Mate — Define, Plan, Identify, Elicit, Analyze, Document, Communicate, Validate, Prioritize, Manage.
BA skills · 14	6 people (listen, interview, communicate, interpersonal, facilitate, lead) + 5 thinking (analytical, systems, on-your-feet, learning, creativity) + 3 craft (observe, organize, model).
BA knowledge	Experience breadth + contemporary RE practices across SDLCs.
BA paths · 4	Former user · developer/tester · project manager · domain expert.

Diagrams you should be able to redraw — three-levels flow (with role labels); feature tree; relationships map (solid = stored in, dashed = origin/influence); RE tree; iterative loop with 5 labeled back-arrows; 7+10 process steps; baseline boundary; effort curves; expectation gap; users⊂customers⊂stakeholders nesting; stakeholder zones; BA bridge (six lanes).

MEMORY HOOK Night-before drill: cover the right column of each table, read the left, recall aloud. Then redraw the twelve figures from the trigger list above.

5.2 Practice question bank

SELF-TEST · WITH ANSWER POINTERS

Answer on paper first; each pointer tells you where to check yourself. If a list question stalls, recite the mnemonic before peeking.

A. Definitions — say it precisely

A1. Define “requirement” completely — include both viewpoints and the three tenses (with the priority attached to each future flavour). *[answer → §1.2]*

A2. Define business, user, and functional requirements — one line each — and name the document each is stored in. *[answer → §1.3, §1.8]*

A3. What is a constraint? Give the slides’ example and one of your own. *[answer → §1.3]*

A4. Define “business rule” and explain why business rules are not themselves software requirements. *[answer → §1.6]*

A5. Define “feature” and reproduce the web-browser example down to two functional requirements. *[answer → §1.7]*

A6. Define stakeholder, customer, and user so the nesting between them is explicit. *[answer → §3.2–§3.3]*

A7. Define “requirements baseline” word-for-word in spirit: what two things must be true of the set, and what does it then serve as? *[answer → §3.7]*

A8. What does an SRS describe, who uses it, and name three things that must stay OUT of it. *[answer → §2.4, §1.9]*

B. Compare & contrast

B1. Product vs. project requirements — definitions plus three concrete project-requirement examples from different G’s. *[answer → §1.9]*

B2. In validation, consistency vs. completeness — define both and give a QuickBite-style violation of each. *[answer → §2.5]*

B3. Direct vs. indirect users — definitions plus the slides’ warehouse example. *[answer → §3.3]*

B4. Requirements development vs. requirements management: what belongs to each, and which artifact marks the border? *[answer → §2.1, §2.7]*

B5. Customer rights vs. customer responsibilities — whose obligations is each list, and which single event appears on both lists from opposite sides? *[answer → §3.4–§3.5]*

B6. Sketch how requirements effort is distributed over time in waterfall, iterative, and agile projects — and state the agile time box the slide mentions. *[answer → §2.6]*

C. List recall — counts must match

C1. The ten classic requirements problems, grouped by their four stages. *[answer → §1.1]*

C2. Five typical sources of business requirements — and the warning about multiple sources. *[answer → §1.3]*

C3. The seven categories of project requirements (recite the 7 G’s with at least two concrete items each). *[answer → §1.9]*

C4. The four elicitation activities. *[answer → §2.2]*

C5. The five analysis activities. *[answer → §2.3]*

C6. The four validation moves, plus the three properties reviews check requirements for. *[answer → §2.5]*

C7. All five back-arrows of the iterative framework — label and direction. *[answer → §2.6]*

C8. The seven requirements-management activities, sorted into the 3 C’s. *[answer → §2.7]*

C9. Six causes of bad requirements AND the ten benefits of a high-quality process (7 fewer + 3 wins). *[answer → §2.9–§2.10]*

C10. All nine customer rights, then all nine customer responsibilities — mnemonics allowed. *[answer → §3.4–§3.5]*

C11. The four decision-making methods in order of widening inclusion, and the four groups whose agreement is needed (with what each group checks). *[answer → §3.6]*

C12. The five groups that gain confidence from a baseline — and what confidence each gains. *[answer → §3.7]*

C13. The BA's ten tasks; the fourteen skills by chunk; the four backgrounds BAs come from. *[answer → §4.2, §4.3, §4.5]*

D. Draw from memory

D1. The three-levels figure — clouds, three ovals, and every role label on the arrows. *[answer → §1.3]*

D2. The relationships map — the three document "shelves," what solid vs. dashed arrows mean, and all four things business rules influence. *[answer → §1.8]*

D3. The requirements-engineering tree. *[answer → §2.1]*

D4. The iterative E-A-S-V loop with its five labeled back-arrows. *[answer → §2.6]*

D5. The baseline-boundary figure — who feeds requirements in, the four-verb oval, both baseline arrows, and both change inputs. *[answer → §2.7]*

D6. The expectation-gap figure AND the BA bridge with all six lane labels. *[answer → §3.1, §4.1]*

E. Scenarios — apply it

E1. Classify each statement for a hospital app as business req / user req / functional req / NFR / constraint / business rule: (a) "cut readmissions 15% in a year" (b) "As a nurse, I want to see a patient's allergies at a glance" (c) "the system shall alert on drug interactions" (d) "patient chart loads in ≤2 s" (e) "must use the hospital's existing Oracle database" (f) "by law, records are retained 7 years." *[answer → §1.3–§1.6]*

E2. "The app should respond quickly and be user-friendly." Name the defect, the problem-cause letter it maps to in CIAO GI, and rewrite it as two testable requirements. *[answer → §2.5, §2.9]*

E3. Two weeks after baselining, marketing demands "scheduled orders." Walk the request through requirements management end-to-end, naming each activity you touch. *[answer → §2.7]*

E4. For a school-bus tracking app, name eight stakeholders covering all three zones — include one indirect user and one regulatory body. *[answer → §3.2–§3.3]*

E5. Your team wants to baseline v1.0 today. Which four groups must agree first, what does each verify — and which decision method would you run the meeting on? *[answer → §3.6]*

E6. Write an acceptance criterion (Given / When / Then) for: "As a customer, I want to reorder my last meal in one tap." Which validation move is this, and who should help establish it? *[answer → §2.5, §3.5]*

MEMORY HOOK Retrieval beats re-reading: one full pass of this bank, closed-book, is worth three passes of highlighting. Wrong answers → follow the pointer → re-answer tomorrow.